

Checklist des Améliorations - Système de Computing Volontaire

☒ Vue d'ensemble

Cette checklist détaille toutes les améliorations à apporter au système de computing volontaire, organisées par priorité et complexité.

☒ 1. Unification Agent Volontaire + Collecteur de Données (Serge Noah)

1.1 Fusion des composants

- ☐ Analyser l'architecture actuelle
 - ☐ Identifier les points d'intégration entre volontaire et collecteur
 - ☐ Mapper les flux de données existants
 - ☐ Évaluer les conflits potentiels
- ☐ Créer l'agent unifié
 - ☐ Concevoir l'architecture du nouvel agent
 - ☐ Implémenter le moteur d'exécution Docker intégré
 - ☐ Intégrer le système de collecte de métriques
 - ☐ Créer le module de rapport en temps réel
- ☐ Tests d'intégration
 - ☐ Tester l'exécution de tâches avec collecte simultanée
 - ☐ Valider la performance (overhead acceptable)
 - ☐ Tester la robustesse en cas d'erreur

1.2 Monitoring de santé

- ☐ Système de health checks
 - ☐ Implémenter les vérifications périodiques
 - ☐ Créer les métriques de santé de l'agent
 - ☐ Implémenter l'auto-diagnostic
 - ☐ Créer le système d'alertes

☒ 2. Système de Choix de Projets

2.1 Interface utilisateur

- ☐ Conception UI/UX
 - ☐ Créer les maquettes de l'interface de sélection
 - ☐ Définir l'expérience utilisateur optimale
 - ☐ Implémenter l'interface web responsive
- ☐ Catalogue de projets
 - ☐ Créer le système d'affichage des projets disponibles
 - ☐ Implémenter les filtres et la recherche
 - ☐ Ajouter les descriptions détaillées des projets
 - ☐ Créer le système de catégorisation

2.2 Système de préférences

- ☐ Gestion des préférences
 - ☐ Créer le modèle de données des préférences
 - ☐ Implémenter la persistance des choix
 - ☐ Créer l'interface de gestion des préférences
 - ☐ Implémenter l'historique des participations
- ☐ Algorithme de correspondance

- ☐ Développer l'algorithme projet↔volontaire
 - ☐ Implémenter le système de scoring
 - ☐ Créer le moteur de recommandations
 - ☐ Tester et optimiser les suggestions
-

❏ 3. Transformation en Service Daemon

3.1 Services système

- ☐ **Service Linux (systemd)**
 - ☐ Créer le fichier .service
 - ☐ Implémenter la gestion des signaux système
 - ☐ Configurer le démarrage automatique
 - ☐ Tester les opérations start/stop/restart
- ☐ **Service Windows**
 - ☐ Créer le service Windows
 - ☐ Implémenter l'interface de contrôle Windows
 - ☐ Configurer l'auto-démarrage
 - ☐ Tester la compatibilité multi-versions Windows

3.2 Configuration et contrôle

- ☐ **Système de configuration**
 - ☐ Créer les fichiers de configuration (YAML/JSON)
 - ☐ Implémenter le rechargement à chaud
 - ☐ Créer les outils de validation de config
 - ☐ Documenter toutes les options
 - ☐ **Interface de contrôle**
 - ☐ Créer l'utilitaire CLI de contrôle
 - ☐ Implémenter les commandes status/logs/config
 - ☐ Créer l'interface web d'administration locale
 - ☐ Ajouter les métriques de monitoring
-

❏ 4. Documentation Complète

4.1 Guide utilisateur

- ☐ **Installation et configuration**
 - ☐ Guide d'installation multi-plateforme
 - ☐ Tutoriel de première configuration
 - ☐ Guide de dépannage courant
 - ☐ FAQ détaillée
- ☐ **Utilisation quotidienne**
 - ☐ Guide d'utilisation de l'interface
 - ☐ Gestion des projets et préférences
 - ☐ Monitoring et rapports
 - ☐ Gestion des erreurs et incidents

4.2 Documentation technique

- ☐ **Architecture et développement**
 - ☐ Documentation de l'architecture système
 - ☐ Guide de contribution au code
 - ☐ Documentation des APIs
 - ☐ Standards de codage et tests
- ☐ **Administration**

- ☐ Guide de déploiement en production
 - ☐ Configuration avancée
 - ☐ Monitoring et maintenance
 - ☐ Sécurité et bonnes pratiques
-

☒ 5. Processus d'Inscription Amélioré

5.1 Détection automatique

- ☐ **Analyse système**
 - ☐ Implémenter la détection des capacités hardware
 - ☐ Créer le profiling automatique des performances
 - ☐ Détecter l'environnement réseau
 - ☐ Évaluer la compatibilité Docker

5.2 Assistant d'onboarding

- ☐ **Interface guidée**
 - ☐ Créer l'assistant pas-à-pas
 - ☐ Implémenter les tests de connectivité
 - ☐ Ajouter la sélection de projets initiaux
 - ☐ Créer le tutoriel interactif
 - ☐ **Configuration automatique**
 - ☐ Auto-configuration des services
 - ☐ Génération automatique des configs
 - ☐ Test de l'installation complète
 - ☐ Validation de la configuration
-

☒ 6. Système de Timing des Tâches

6.1 Collecte des délais

- ☐ **Timestamps d'exécution**
 - ☐ Implémenter la capture de l'heure de réception
 - ☐ Enregistrer l'heure de début d'exécution
 - ☐ Capturer l'heure de fin de tâche
 - ☐ Enregistrer l'heure de soumission du résultat
- ☐ **Gestion des statuts**
 - ☐ Implémenter le tracking du statut de fin
 - ☐ Créer le système d'informations de tâche
 - ☐ Développer le logging des checkpoints
 - ☐ Implémenter la gestion pause/reprise

6.2 Système de checkpoints

- ☐ **Mécanisme de sauvegarde**
 - ☐ Implémenter les points de contrôle automatiques
 - ☐ Créer la gestion des pauses manuelles
 - ☐ Développer la reprise de tâches
 - ☐ Tester la robustesse du système
-

☒ 7. Collecte de Données Statiques

7.1 Informations système de base

- ☐ **Identité et OS**

- ☐ Capturer la version de l'agent
 - ☐ Enregistrer le nom d'hôte
 - ☐ Détecter l'OS (nom, version, architecture)
 - ☐ Implémenter la détection du type PC (laptop/desktop)
- ☐ **Spécifications hardware**
 - ☐ Détecter le nombre de CPU (physiques/logiques)
 - ☐ Capturer les fréquences CPU (min/max)
 - ☐ Mesurer la mémoire totale et swap
 - ☐ Calculer l'espace disque total

7.2 Informations avancées

- ☐ **Réseau et périphériques**
 - ☐ Capturer les adresses MAC
 - ☐ Détecter les informations GPU
 - ☐ Enregistrer les infos BIOS/UEFI
 - ☐ Capturer les informations carte mère

8. Collecte de Données Dynamiques

8.1 Métriques de performance

- ☐ **CPU et mémoire**
 - ☐ Implémenter le monitoring CPU par core
 - ☐ Surveiller l'utilisation mémoire détaillée
 - ☐ Tracker l'utilisation swap
 - ☐ Mesurer les fréquences CPU actuelles
- ☐ **Stockage et réseau**
 - ☐ Monitorer l'utilisation disque
 - ☐ Tracker l'activité réseau (octets/paquets)
 - ☐ Implémenter la mesure de température CPU

8.2 Système de collecte périodique

- ☐ **Configuration de la collecte**
 - ☐ Implémenter la collecte par intervalle (5 min)
 - ☐ Créer la collecte par seuil de données (200Ko)
 - ☐ Développer le système de timestamps
 - ☐ Implémenter la détection de démarrage/arrêt système

9. Intégration et Tests

9.1 Tests d'intégration

- ☐ **Tests de bout en bout**
 - ☐ Tester l'installation complète windows et linux
 - ☐ Valider le processus d'onboarding
 - ☐ Tester l'exécution de tâches avec collecte
 - ☐ Valider la persistance des données
 - ☐ Valider la reprise des tâches après le démarrage de l'OS

9.2 Tests de performance

- ☐ **Benchmarks**
 - ☐ Mesurer l'overhead de collecte
 - ☐ Tester la scalabilité
 - ☐ Valider la stabilité long terme
 - ☐ Optimiser les performances critiques

☒ 10. Monitoring et Métriques

10.1 Dashboard de monitoring

- ☐ **Interface de suivi**
 - ☐ Implémenter les métriques temps réel
 - ☐ Ajouter les alertes et notifications
 - ☐ Créer les rapports de performance (À discuter)

10.2 Analytics et reporting

- ☐ **Analyse des données**
 - ☐ Implémenter l'analyse des patterns d'usage des volontaire
 - ☐ Créer les rapports de performance historiques
 - ☐ Développer les métriques de qualité de service auprès des clients

☒ 11. Fonctionnalités du Coordinateur (Ashley et Patrice)

11.1 Validation des clients

- ☐ **Système de validation**
 - ☐ Implémenter la vérification des inscriptions clients
 - ☐ Développer les alertes pour les connexions invalides
 - ☐ Tester la robustesse contre les tentatives frauduleuses

11.2 Gestion des clients

- ☐ **Outils de gestion**
 - ☐ Implémenter les actions d'activation/désactivation
 - ☐ Ajouter le monitoring individuel des clients
 - ☐ Développer les rapports sur l'activité des clients (plutard)

11.3 Gestion des workflows en temps réel (RUD, Stop, Reprise)

- ☐ **Opérations sur workflows**
 - ☐ Implémenter la lecture (Read) des workflows existants
 - ☐ Créer les fonctionnalités de mise à jour (Update) et suppression (Delete)
 - ☐ Développer les commandes de stop et reprise des workflows
 - ☐ Tester la gestion des états intermédiaires et erreurs

11.4 Gestion des tâches en temps réel(RUD, Stop, Reprise)

- ☐ **Opérations sur tâches**
 - ☐ Implémenter la lecture (Read) des tâches en cours et historiques
 - ☐ Créer les fonctionnalités de mise à jour (Update) et suppression (Delete)
 - ☐ Développer les commandes de stop et reprise des tâches
 - ☐ Intégrer la gestion des dépendances entre tâches

11.5 Correction de la fonctionnalité temps réel

- ☐ **Migration vers WebSockets**
 - ☐ Analyser l'implémentation actuelle avec requêtes périodiques
 - ☐ Implémenter les WebSockets pour les mises à jour en temps réel
 - ☐ Tester la réduction de latence et la fiabilité
 - ☐ Optimiser la consommation de ressources côté coordinateur

☒ 12. API pour la Présentation des Performances sur le Site Internet (Morel et Serge Yanick)

12.1 API pour les Performances

- ☐ **Endpoints de performances**
 - ☐ Créer l'API pour récupérer les métriques globales (CPU, mémoire, etc.)

- ☐ Implémenter les filtres par période (jour, semaine, mois)
- ☐ Ajouter les agrégations (moyennes, pics, tendances)
- ☐ Tester la sécurité et l'authentification des accès

12.2 API pour les Workflows

- ☐ **Endpoints workflows**
 - ☐ Implémenter la liste des workflows avec détails (statut, durée)
 - ☐ Créer les endpoints pour filtrer par statut ou projet
 - ☐ Ajouter les historiques et logs associés
 - ☐ Intégrer des options de pagination et recherche

12.3 API pour les Tâches

- ☐ **Endpoints tâches**
 - ☐ Créer l'API pour lister les tâches avec métriques (temps d'exécution, statut)
 - ☐ Implémenter les filtres par volontaire ou workflow
 - ☐ Ajouter les détails sur les erreurs et reprises
 - ☐ Développer les endpoints pour les statistiques agrégées

12.4 API pour les Volontaires (avec recherche par ID)

- ☐ **Endpoints volontaires**
 - ☐ Implémenter la recherche par ID spécifique
 - ☐ Créer la liste des volontaires avec profils (hardware, activité)
 - ☐ Ajouter les filtres par performance ou connexion
 - ☐ Intégrer la confidentialité des données sensibles

12.5 API pour les Badges

- ☐ **Endpoints badges**
 - ☐ Créer les calculs automatiques pour badges (volontaire de la semaine, mois, année)
 - ☐ Implémenter les catégories : plus connecté, plus rapide, plus de tâches
 - ☐ Développer les endpoints pour afficher les badges attribués
 - ☐ Ajouter les notifications et historiques des badges

12.6 API pour les Données Collectées et leur Exploitation

- ☐ **Endpoints données collectées**
 - ☐ Implémenter l'accès aux données statiques et dynamiques
 - ☐ Créer des outils d'exploitation (Pour plutard)
 - ☐ Ajouter les agrégations pour rapports (par volontaire, projet .. Pour plutard)

☒ Critères de Validation

Fonctionnalité

- ☐ L'agent unifié fonctionne sans dégradation de performance
- ☐ Le système de choix de projets est intuitif et efficace
- ☐ Le service daemon démarre automatiquement et se gère proprement
- ☐ La documentation couvre tous les cas d'usage
- ☐ L'onboarding se fait en moins de 10 minutes
- ☐ Toutes les métriques sont collectées correctement
- ☐ Les fonctionnalités du coordinateur gèrent correctement clients, workflows et tâches
- ☐ Les API du site internet présentent les données de manière sécurisée et performante

Performance

- ☐ Overhead de collecte < 5% CPU et mémoire
- ☐ Temps de réponse interface < 2 secondes
- ☐ Démarrage du service < 30 secondes
- ☐ Pas de fuite mémoire sur 24h de fonctionnement
- ☐ Latence des WebSockets < 1 seconde pour mises à jour temps réel

Robustesse

- ☐ Récupération automatique après crash
- ☐ Gestion gracieuse des erreurs réseau
- ☐ Sauvegarde des données en cas d'arrêt inattendu
- ☐ Compatibilité multi-plateforme validée
- ☐ Validation des clients empêche les accès non autorisés

Date de création : 2 septembre 2025 Dernière mise à jour : 27 septembre 2025